CS5187 Term Project

Directed LLM-Guided Fuzzing of DuckDB

Team guide: what we are doing, every term explained, how the folders and files fit together, who does what, what we have actually achieved, and how to explain it all in a meeting.

Team	Prabhnoor, Niket, Jahnvi
Target software	DuckDB (a widely used analytical database, written in C++)
Goal that earns full marks	Find a previously unknown bug that DuckDB's maintainers accept
Code repository	github.com/PASS-IITG/cs5187-term-project-janhvimittal07
Document status	Written 15 Oct 2026, the day of progress meeting 1. Every number comes from a run in this project unless it says it is a teammate's claim or a published paper's claim.

How to use this document. Section 1 is the 2-minute version. Section 3 is the dictionary: when any word in the other sections is unclear, look it up there. Section 8 is the meeting preparation. Nothing here assumes you know anything about fuzzing, databases or compilers.

Contents

- 1. The project in plain words
- 2. How the whole system works (the pipeline diagram)
- 3. Glossary: every term in one place
- 4. Folder and file map (what every file is for)
- 5. Who does what (work split) and how far each person has got
- 6. What we have achieved so far (results, honestly stated)
- 7. How to run things (commands you can copy)
- 8. Preparing for the progress meetings (talk outline, questions and answers)
- 9. Plan, risks and open decisions
- 10. Appendix: upstream references, papers, and what has not been verified

1. The project in plain words

The one-sentence version. We are building a tool that automatically writes lots of unusual database queries, runs them against DuckDB, and notices when DuckDB crashes or gives a wrong answer, so that we can report a genuine bug to DuckDB's developers.

What is DuckDB?

DuckDB is a popular, free, open-source database. Programs (and data scientists) send it questions written in a language called SQL, for example `SELECT count(*) FROM sales WHERE price > 10;`, and it answers them. Because it is used by many people, a bug in it matters, and its developers have a public process for handling bug reports. That is why the course allows it as a target.

What is 'fuzzing' and why is it hard here?

Fuzzing means throwing huge numbers of strange inputs at a program to see if it breaks. For a database the input is SQL. The difficulty: if you generate SQL at random, DuckDB rejects almost all of it immediately ('column not found', 'syntax error') and the deep parts of the database are never exercised. Our measurement confirms this: about half of the randomly mutated queries we generated failed before reaching anything interesting.

What is new in our approach?

- We read DuckDB's own source code and list about 25,000 'interesting spots' (boundaries, risky operations).
- We measure which code the queries actually execute and find the large parts that nothing has ever run.
- We ask a language model (an LLM) to write SQL aimed at those unexecuted parts, check it is valid, run it on special builds of DuckDB that shout when memory is misused, and compare behaviour across settings.
- Everything after that (shrinking a failing query, checking whether the developers already know about it) is automated, so a person only reads a short report.

What does the course want?

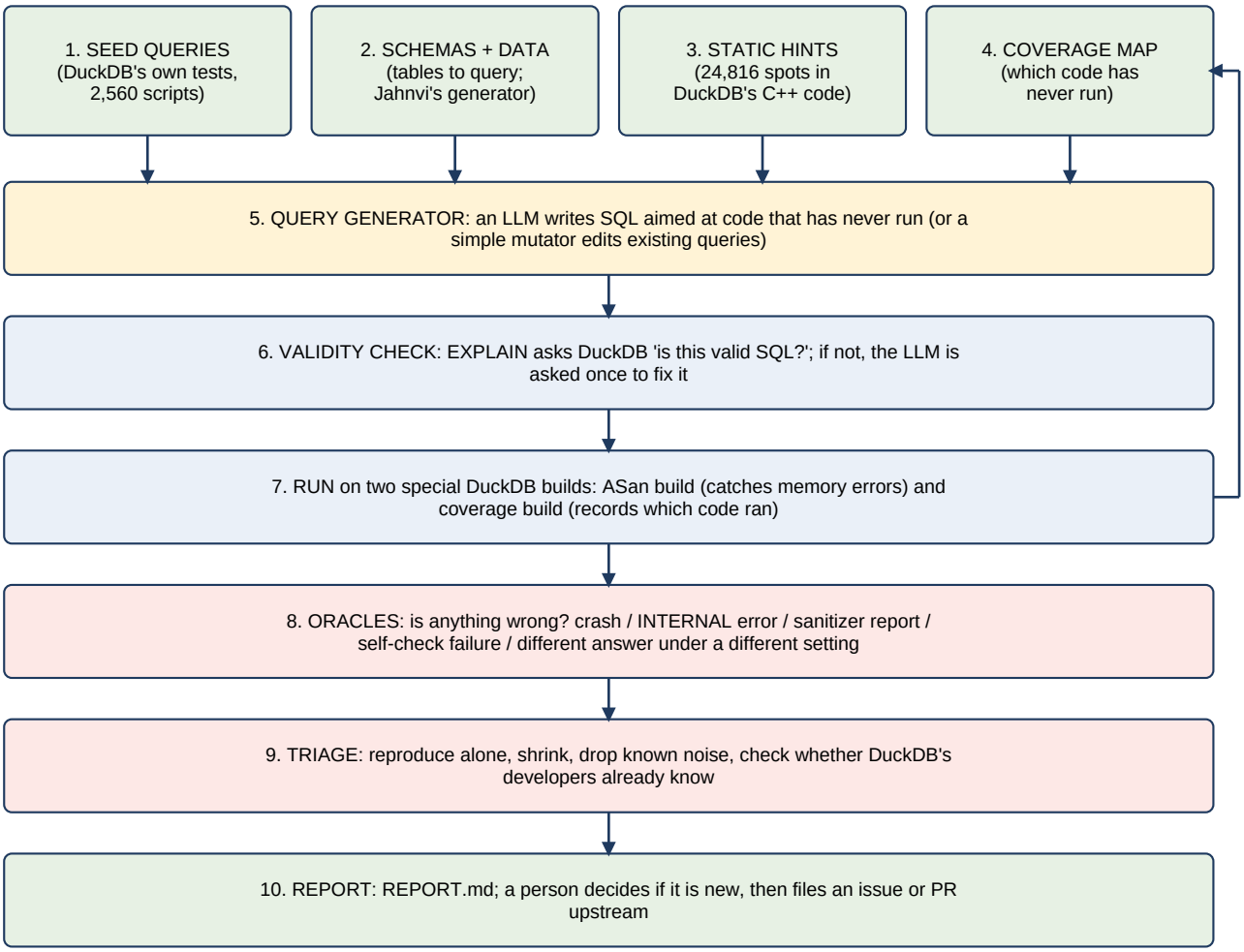
Component	Weight	What it needs
Implementation and results	50%	A real, previously unknown bug that upstream maintainers accept (merged PR, accepted CVE or acknowledged advisory) gets full marks. Good tooling and reproducible evaluation earn partial credit even without a bug.
Progress meetings + final presentation	25%	3 progress meetings and the final talk. Every member must be able to answer any question about any part. 'That was X's part' is not acceptable.
Report (FSE format)	25%	Motivation, technique, implementation, evaluation, related work, threats to validity. Each member signs a statement of their contribution; commit history is checked.

Date	Milestone	What is expected
11-16 Oct 2026	Progress meeting 1 (today)	Scope firm, target accessible, initial pipeline running
26-30 Oct 2026	Progress meeting 2	First end-to-end results, first bug candidates, evaluation plan
2-6 Nov 2026	Progress meeting 3	Upstream reports or PRs filed, results consolidated, report draft
Thu 12 Nov 2026	Final presentation	10-15 minutes plus questions
Fri 13 Nov 2026, 23:59 IST	Paper + repo due	Overleaf link, PDF, public GitHub repo, reproduction instructions

Course rule on AI tools: LLM use must be disclosed and cannot replace your own understanding. Much of the code in `src/harness`, `src/llm` and `src/coverage` was written with Claude (an AI assistant) under Prabhnoor's direction. Disclose this plainly, and each person should be able to explain the files they own.

2. How the whole system works

Read the diagram top to bottom. The top row is raw material that is prepared once. The generator produces SQL, the SQL is checked and run on special builds, oracles decide whether anything went wrong, and triage turns raw alarms into a short report. The feedback loop re-measures coverage so the next round aims at code still untouched.



FEEDBACK LOOP: after every round the coverage map (4) is recomputed from the new runs, so the next prompts aim at code still untouched. Preparation (items 1-4) is done once; steps 5-10 repeat.

The same flow as a numbered story

- **Preparation (done once).** Extract seed queries from DuckDB's test suite; extract hints from its C++ source; generate sample tables; build DuckDB twice (an ASan build and a coverage build).
- **Generation.** For a chunk of code that has never run, build a prompt (tables + the C++ snippet) and ask the model for SQL. Throw away anything DuckDB cannot even parse; ask the model to repair failures once.
- **Execution.** Run the SQL on the ASan build (to catch crashes and memory errors) and on the coverage build (to learn what code it reached). Run it under several settings (1 thread vs 8, optimizer on vs off) to compare answers.
- **Judging.** The oracles flag: a crash, an 'INTERNAL Error', a sanitizer report, a verification mismatch, or a different answer under a different setting.
- **Triage.** Automatically re-run the suspect alone, shrink it to the smallest query that still fails, remove known-harmless patterns, and search DuckDB's GitHub for the same bug. What remains goes in REPORT.md.
- **Human step.** A person decides whether it is genuinely new. Only then do we write to the maintainers.

A very important lesson from our own results

Every real bug our tool found in DuckDB v1.5.6 turned out to be already reported or already fixed in newer code. The released version lags the development code by months. So the target for finding a *new* bug must be the latest development code (called 'main'), not the release. Section 6 explains the evidence.

3. Glossary: every term in one place

Grouped by topic. Each entry says what it is, in plain words, and why it matters for us.

3.1 The setting

Term	Plain-words meaning (and why we care)
Bug	A mistake in a program that makes it behave wrongly: crash, wrong answer, or a safety check failing. Our whole goal is to find one nobody knew about.
Open source (OSS)	Software whose code is public. Anyone can read it and propose changes. DuckDB is open source.
Upstream	The project's official home (for us github.com/duckdb/duckdb), as opposed to our private copy. 'Reporting upstream' means telling the official developers.
Maintainers	The people who run the upstream project and decide which reports and changes are accepted.
Issue	A bug report posted on GitHub. Numbered, for example #26300.
Pull request (PR)	A proposed code change, usually a fix, waiting for review. 'Merged' means accepted into the official code. We can only claim a bug 'accepted' when its issue or fix is acknowledged by maintainers.
Duplicate	A bug that has already been reported. Reporting one gains nothing, so we check for duplicates first.
Zero-day / 'previously unknown'	A bug nobody has reported yet. The course needs one of these.
DuckDB	A fast embedded analytical database written in C++. Our target.
SQL / query / statement	SQL is the language for talking to databases. A query is one SQL question, a statement is one SQL command (including those that change data).
Schema / table	A table is a grid of data; the schema is the description of the tables (names, columns, types). A query needs tables to ask questions about.
Optimizer	The part of a database that rewrites your query into a faster plan before running it. Optimizer bugs are a rich source of wrong answers, because the rewritten plan can differ from the original.

3.2 Versions and builds

Term	Plain-words meaning (and why we care)
Release (v1.5.6)	A frozen, published version of DuckDB. This is what most users run and what we started with.
main / development code	The newest unreleased code, changing daily. Bugs found in the release are often already fixed here.
Dev wheel	A ready-made nightly package of the development code that can be installed with pip (we used 2.0.0.dev2610011535). Convenient, but it lacks sanitizers and its self-check is switched off.
Commit / hash / pinned	A commit is one saved change in the code history, identified by a long hash such as 690d113c. 'Pinned' means we always use that exact snapshot so results can be reproduced.
Build / compile	Turning source code into a runnable program. We compile DuckDB ourselves, twice, with different options. A full build takes an hour or more on our laptop.
Stock build	A normal, unmodified build (for instance the pip package). We use it to confirm that a bug is real and not an artifact of our special builds.

3.3 Testing ideas

Term	Plain-words meaning (and why we care)
Fuzzing / fuzzer	Automatically generating many odd inputs to find crashes and wrong behaviour. The program that does it is a fuzzer.
Seed / corpus	A seed is a starting input that is already valid. The corpus is the whole collection. Our seeds are SQL scripts extracted from DuckDB's own tests.
Mutation vs generation	Mutation edits an existing query a little (swap a number, change a join). Generation writes a new query from scratch. We use both.
Baseline	The simple method we compare against, so we can honestly say whether the LLM helps. Ours is a random mutator with no LLM. The proposal also promises OSS-Fuzz and SQLsmith as baselines.
OSS-Fuzz	Google's free service that fuzzes important open-source projects around the clock. DuckDB is enrolled.
SQLsmith	An older tool that generates random but valid SQL. A classic baseline.
Regression test	A test added after a bug was fixed, so the bug cannot come back. DuckDB keeps about 280 such files from past fuzzers in test/fuzzer. They are valuable seeds: bugs cluster near old bugs.
sqllogictest (.test)	DuckDB's test file format: SQL statements plus expected results. We convert these into plain SQL.
Differential testing	Run the same query two different ways (e.g. optimizer on and off) and see if the answers match. A mismatch means one way is wrong.
Oracle	A rule that tells you whether a result is wrong. Without one you only notice crashes. Ours: crash, INTERNAL error, sanitizer report, differential mismatch, DuckDB's verification.
TLP / NoREC	Two published oracles for database testing. TLP splits rows by a condition (true, false, unknown) and checks the parts add up to the whole. NoREC rewrites a query so the optimizer cannot help, then compares. Planned (task N5), not yet built.
False positive / noise	An alarm that is not really a bug. Most of our effort goes into recognising these.
'By design'	Behaviour that looks inconsistent but is intended, for example a function that returns the optimizer's own statistics. Reporting it wastes the maintainers' time.
Non-deterministic	Gives different answers on different runs by nature (random numbers, UUIDs, current time, 'first row' without an ORDER BY). Comparing runs of these produces false alarms.

3.4 What goes wrong, and the tools that notice

Term	Plain-words meaning (and why we care)
Crash / segfault	The program dies abruptly (often a memory error). Always a bug.
Assertion / INTERNAL Error	DuckDB's own safety checks. 'INTERNAL Error' means DuckDB itself detected that something that should be impossible happened. Maintainers treat these as bugs.
Sanitizer: ASan, UBSan	Special compiler options that make a program stop and explain when it misuses memory (AddressSanitizer) or does something the language forbids, like using a null pointer (UndefinedBehaviorSanitizer). Many memory bugs are silent without them. Our 'ASan build'.
Null pointer	A reference to nothing. Using it usually crashes the program.
enable_verification	DuckDB's built-in self-check mode. It runs each query several ways (copied, written to text and re-read, saved and re-loaded, optimizer off) and complains if the answers differ. A powerful free oracle. Important: it is inert on the dev wheel.
Coverage (line / branch)	Which lines or decision branches of DuckDB's code ran. High coverage means more of the program was exercised, which is our main measure of 'how deep did we get'.
Coverage build	A DuckDB compiled to count what executes. Tools llvm-profdata and llvm-cov turn the counts into reports. Files ending .profraw / .profdata are those counters.
Uncovered region	A block of code that no query has run yet. The generator is aimed at these.
Static analysis / hint	Reading code without running it. A hint is a spot worth reaching: a boundary constant (like 2048), a comparison, or a risky memory operation. We extracted 24,816.
tree-sitter / CodeQL	Two tools for analysing source code. CodeQL needs a heavy build; tree-sitter is light. Jahnvi chose tree-sitter and documented why.

Term	Plain-words meaning (and why we care)
Vector size 2048	DuckDB processes rows in blocks of 2048. Bugs often sit at the edges, so 2047, 2048 and 2049 rows are classic test sizes.

3.5 Our pipeline words

Term	Plain-words meaning (and why we care)
Triage	Sorting alarms into: real new bug, already-known bug, or noise. Ours is automated in five steps: (1) reproduce it alone, (2) find the setting that triggers it, (3) shrink it, (4) confirm on a stock build, (5) search upstream issues and PRs.
Minimize / delta debugging	Repeatedly deleting statements while the failure remains, until only a handful are left. A 2-line repro is far easier to understand and report than a 50-line one.
Signature	A short fingerprint of a failure (the error text, or the top of the stack trace) used to count the same bug once instead of 400 times.
Noise gate	A rule that discards known harmless patterns before a human sees them (e.g. queries using random(), side effects, or debug functions).
Determinism gate	Re-run a suspect on one frozen copy of the data. If the difference disappears, the cause was different random data, not a bug.
Artifact / race	A problem caused by our own tooling. Example: parallel workers writing the same temp file.
Pipeline / stage	The chain of steps; fuzz.py runs them: seeds, sweep, baseline, triage, report.
Sweep	Running every script in the corpus once under some configuration and tabulating outcomes.
Hunt	Running the corpus under many verification settings to look for crash-class failures (hunt.py).
Campaign	A long unattended fuzzing run (hours). Task N6, not built yet.
FUZZ_WORK	An environment variable naming the folder where large generated things live (DuckDB source, builds, results). Ours is /home/giraffe/cs5187. It keeps them out of the git repo.
venv	A private Python environment so package versions do not collide.

3.6 LLM words

Term	Plain-words meaning (and why we care)
LLM	A large language model, an AI that continues text. Here it writes SQL.
Local model / Ollama	A model running on our own computer instead of a paid web service. Ollama is the program that runs it. We use qwen2.5-coder:1.5b (1.5 billion parameters, about 1 GB). It is small.
Prompt	The text we send to the model: tables, a code snippet, and the instruction.
Temperature	How adventurous the model is. Low (0.2) gives safer, more valid output. We used 0.8, probably too high.
Few-shot	Putting a few good examples in the prompt. Published work shows this helps a lot.
Validity rate	The share of generated queries DuckDB accepts. A key measure of how useful the model is.
EXPLAIN (validity check)	Asking DuckDB to plan a query without running it. If it plans, the SQL is valid.
Guided vs unguided	Guided prompts show the model a never-executed code snippet to aim at. Unguided prompts just ask for varied queries. Comparing them tests whether 'directing' helps.
Plateau	Coverage stops growing because the model keeps producing similar queries. Seen in our unguided run.

3.7 Project and git words

Term	Plain-words meaning (and why we care)
RQ (research question)	A question the paper answers with data, e.g. RQ1: does directing the LLM increase coverage?
Threats to validity	An honest section listing why our results might be wrong (small model, small sample, one target).
Git / commit / push / pull	Git records history. Commit saves a change locally; push uploads to GitHub; pull downloads teammates' changes. Commit history is part of your grade.
WORKPLAN.md	docs/WORKPLAN.md: the shared plan with task IDs (N1.., J1.., P1..) and owners.
Progress meeting	A checkpoint with the instructor. Three of them, then the final presentation.

4. Folder and file map

There are two places on the laptop. The **git repository** (what gets graded and shared) holds only code and small documents. The **work folder** holds the big generated things and is never committed.

```
~/term-project/                 <- THE GIT REPO (shared on GitHub)
|-- README.md                   course instructions (from the template)
|-- docs/
|   |-- WORKPLAN.md             plan: tasks, owners, interfaces
|   `-- PROJECT_GUIDE.pdf       this document
|-- paper/                      the FSE report (LaTeX skeleton)
|   |-- sections/*.tex          one file per paper section
|   |-- data/                   raw numbers behind every table (RQ3 files so far)
|   |-- HANDOFF.md              numbers table + sources, keep current
|   |-- CHECKLIST.md, STYLE.md  submission rules
|   `-- Makefile                `make` builds the PDF
`-- src/                        ALL CODE
    |-- build.sh  setup.sh      build DuckDB v1.5.6 (ASan + coverage); one-time setup
    |-- build-main.sh           build a pinned commit of DuckDB main (Niket)
    |-- DUCKDB_VERSION          the pinned release (v1.5.6)
    |-- harness/                the testing machinery (section 4.1)
    |-- coverage/cov.py         coverage measurement and gap reports
    |-- static/                 static analysis: hints (Jahnvi)
    |-- gen/                    sample tables and data generator (Jahnvi)
    |-- llm/                    the LLM client, generator and loop (Prabhnoor)
    `-- docs/                   Niket's validation notes

/home/giraffe/cs5187/           <- FUZZ_WORK (big, NOT in git)
|-- duckdb/                     DuckDB v1.5.6 source
|-- build-asan/duckdb           the ASan+UBSan build (finds memory errors)
|-- build-cov/duckdb            the coverage build (records what ran)
|-- corpus/seeds-v2/            2,560 seed scripts (newest extraction)
|-- corpus/llm/{G1,U1}/         scripts the LLM generated in the pilot
|-- gen_out/                    5 sample schemas
|-- results/                    logs, triage output, hunt output, REPORT.md
|-- .venv/  .venv-dev/          Python environments (release / dev wheel)
`-- wheels/                     downloaded dev wheel
~/ollama/                       the local LLM runtime and model
```

4.1 What every important file does

File	What it does, in plain words	Owner	Status
harness/runner.py	Runs a SQL script in a DuckDB build with a time limit and labels what happened: ok, syntax error, internal error, sanitizer report, crash, timeout. Everything else builds on this.	Niket	Done
harness/extract_seeds.py	Converts DuckDB's test files into plain SQL scripts (our seed corpus). Jahnvi extended it: loop unrolling, 2,560 files / 42,637 statements.	Jahnvi	Done
harness/oracle.py	Differential checks: same query under 5 settings (default, optimizers off, 1 thread, 8 threads, no insertion order), compare answers; also a TLP helper.	Niket	Done (TLP/NoREC generators = N5, todo)
harness/sweep_seeds.py	Runs every seed once on the ASan build and counts outcomes.	Niket	Done

File	What it does, in plain words	Owner	Status
harness/sweep_oracle.py	Replays every seed under the 5 settings in lockstep and records any difference.	Niket	Done; noise gates partial (N8)
harness/triage.py	The automatic triage: reproduce, find triggering setting, shrink, confirm on stock build, search upstream issues and PRs, run the pinned-main check.	Niket	Done
harness/main_triage.py	Stand-alone check of a SQL file against the pinned main build.	Niket	Done
harness/baseline.py	The no-LLM baseline: randomly mutates seeds (boundary values, join swaps, nested wrapping) and tracks which mutator works.	Jahnvi	Done
harness/fuzz.py	One command that runs all stages and writes REPORT.md.	Prabhnoor	Done
harness/hunt.py	Runs all inputs under DuckDB's verification modes and selected optimizer switches; keeps only crash-class results; shrinks and checks upstream. Has a guard against builds with inert verification.	Prabhnoor	Done
harness/devcheck.py	Installs the newest dev wheel and checks whether a finding still fails there.	Prabhnoor	Done
harness/smoke_test.py	Quick self-test that the tools work.	Niket	Done
coverage/cov.py	Merges coverage across runs, lists never-executed regions and hints, per-function reports.	Jahnvi	Done
static/extract_hints.py	Reads DuckDB's C++ with tree-sitter and writes hints.json (24,816 hints).	Jahnvi	Done
static/eval_hints.py	Measures how many hints the seed corpus actually executes (RQ3 data).	Jahnvi	Done
static/APPROACH.md	Why tree-sitter was chosen over CodeQL and Clang; limits.	Jahnvi	Done
gen/generate_schema.py	Creates 5 sample databases: extreme values, nested types, 2047/2048/2049/4096-row tables, NULL patterns, relational tables.	Jahnvi	Done
llm/client.py	Talks to a local model server over HTTP. Settings via environment variables.	Prabhnoor	Done
llm/generate.py	Builds the prompt, asks the model, checks validity with EXPLAIN, repairs once, saves scripts and metrics.	Prabhnoor	Done
llm/loop.py	Coverage-guided loop: target unexecuted code, generate, run, re-measure. --unguided gives the control arm.	Prabhnoor	Done
llm/install_ollama.sh	Installs the local model runtime without needing admin rights.	Prabhnoor	Done
llm/test_*.py	Automated tests of the client, generator and loop using a fake model server.	Prabhnoor	Done
src/docs/*.md	Niket's evidence: build flags, build-time measurements, pinned-main validation, Week 1 run.	Niket	Done

Not yet in the repo: src/driver (faster in-process runner, N3), src/baselines (OSS-Fuzz and SQLsmith, N4), harness/campaign.py (long-run manager, N6), src/eval (evaluation scripts, P5). These are on the plan, not built.

5. Who does what

The original proposal split the work by area. In October we rebalanced it so the three loads are within one point of each other (effort points: small 1, medium 2, large 3). The shared plan is docs/WORKPLAN.md.

	Niket	Jahnvi	Prabhnoor
Theme	Execution, oracles, baselines, campaigns	Program analysis, coverage gaps, test data	LLM generation and the feedback loop
Core task	N3: faster in-process driver	J2: extract hints	P2: generator with validity repair
Evaluation share	RQ4: bugs found; OSS-Fuzz/SQLsmith baselines	RQ3: effect of static hints	RQ1/RQ2: coverage, valid-query rate
Plan points	15	14	15

5.1 Task list and status (as of 15 Oct 2026)

Task	What it is	Status
N1	Review build flags, document build time and RAM	DONE (cold single-file timings; no whole-build wall-clock)
N2	Pinned main build + 'reproduces on main?' check	DONE, validated on the real ASOF and MERGE bugs
N3	In-process batch driver (speed)	NOT STARTED
N4	OSS-Fuzz and SQLsmith baselines	NOT STARTED (promised in the proposal)
N5	TLP / NoREC oracles	NOT STARTED
N6	Long-run campaign runner	NOT STARTED
N8	Noise gates for sweeps and triage	PARTIAL (about one third)
J1	Choose analysis approach	DONE (tree-sitter, written up)
J2	Extract hints	DONE (24,816 hints)
J3	Coverage tooling and hint cross-reference	DONE
J4	Schema and data generator	DONE (5 schemas, all validate)
J5	Better seed extraction	DONE (2,560 files / 42,637 statements; numbers not yet written in a doc)
J6	Better mutators and effectiveness table	DONE
J7	Hint evaluation (RQ3)	DONE (paper/data, paper table)
P1	LLM client	DONE (pushed)
P2	Generator with EXPLAIN repair	DONE (pushed)
P3	Coverage feedback loop	DONE (pushed)
P4	Prompt strategy experiments	NOT STARTED (needs a stronger model or better prompts)
P5	Evaluation harness (equal budgets, plots)	NOT STARTED (baseline arm data exists)
P6	Paper integration	NOT STARTED
P7	Metrics logging	PARTIAL (rows logged; no shared schema document)
P8	Dockerfile / setup doc	NOT STARTED

5.2 Honest summary of the split

- **Jahnvi:** all seven tasks done, and the strongest paper-ready result (RQ3).
- **Prabhnoor:** the LLM stack (client, generator, loop), the one-command pipeline, the bug-hunt tooling and the dev-build check are built and pushed. Evaluation and prompt experiments are still open.
- **Niket:** Week 1 complete after two review rounds. The four big tasks (N3 to N6) have not started, and N4 (OSS-Fuzz baseline) is the largest open risk because the proposal promises it.

Everyone must answer questions about everything. Before each meeting, each person should spend ten minutes learning a part that is not theirs. Section 8 has the questions.

6. What we have achieved (results, honestly stated)

Headline: we have a working end-to-end pipeline and a clear picture of the problem, but no new bug yet. The tool did find genuine bugs in DuckDB v1.5.6; every one was already known or fixed upstream.

6.1 Experiments and numbers

Experiment	Result	What it means
Seed extraction	Old extractor: 1,959 files / 26,981 statements. New (Jahnvi): 2,560 files / 42,637 statements (+58%). Skips fell from 1,454 to 853 files.	More and richer starting material
Crash sweep (ASan) of new seeds	2,560 scripts in about 3.5 minutes: 1,154 ran cleanly (45%), 748 bind errors, 486 runtime errors, 170 syntax errors, 1 timeout, 1 crash (most likely a harness race: it does not reproduce when run alone).	Classifier works on realistic traffic
Differential sweep (old seeds, 5 settings)	11,235 queries compared. Two real crashes surfaced; both already known upstream.	The method finds real bugs
Mutation baseline (no LLM), 10 min	Coverage of the scanned code climbed to about 46% of lines by run 250 (different denominator than the LLM runs).	The baseline is fast-growing; the LLM must beat this
Static hints (Jahnvi)	24,816 hints (12,580 branches, 9,845 constants, 2,391 sinks). Seed corpus executes 19.1%; 80.9% never run (sinks 78.5%, branches 79.0%, constants 84.1%).	Strong motivation for directed generation (teammate's number, not independently re-run)
LLM pilot: guided vs unguided	Guided: 72 of 126 valid (57%), +4,645 lines, 0 of 10 targets reached. Unguided: 74 of 97 valid (76%), +7,728 lines.	With a 1.5B model, aiming at C++ did not help (small sample)
Bug hunt on v1.5.6 (ASan)	14,000 of 18,665 runs: 109 hits in 22 unique signatures; all explained (6.2).	No new bug; three real old ones (the MERGE bug came from the earlier differential sweep)

6.2 The real bugs we found (sweeps and hunt), and why none counts

Bug	What happens	Upstream status	Counts?
ASOF join crash	Segfault on an ASOF join with a non-comparison condition when the join-order optimizer is disabled (null pointer).	Issue #26300, fixed by PR #26301 (merged 8 Oct 2026)	No
MERGE INTO internal error	INTERNAL Error 'Failed to bind column reference rowid' for MERGE ... WHEN NOT MATCHED BY SOURCE with build_side_probe_side disabled.	Fix PR #26715 merged 12 Oct 2026	No
COLLATE inside BETWEEN	DuckDB prints a query back to text and re-reads it as a self-check; the printed text drops parentheses and no longer parses.	Same class reported in 2023 (#5984, #7128); parser now accepts it on the dev build	No
Prepared statement + COLLATE NOCASE	contains(b COLLATE NOCASE, $1) returns no rows with filter_pushdown disabled (wrong answer).	Correct on the newest dev build	No

The other 19 hit signatures were noise: DuckDB's own regression-test inputs (accepted behaviour), functions that legitimately differ under verification (STATS(), current_query(), txid_current(), nextval, first() without ORDER BY, settings tables), an optimizer rule the project's own test expects (IN-list rewrite), and artifacts of our harness (unreplaced placeholders; workers sharing files).

6.3 Lessons that shape the next steps

- **The release lags development.** Every real bug found in v1.5.6 was already fixed. Future hunting must run on the latest code.
- **The dev wheel cannot do verification.** PRAGMA enable_verification does nothing there (we measured: five queries that must trip it passed). Hunting latest code with verification needs our own sanitizer build of main, which Niket already has pinned at 9 Oct 2026.
- **Noise dominates.** 35 of 37 first-run findings were noise. Noise gates (task N8) are essential, not optional.
- **LLM size and prompt design matter.** Published work (WhiteFox, MIST) says to feed the model a plain-language requirement derived from the code, not raw C++, to show it queries that worked, and to use low temperature. We did none of those yet.
- **Coverage is a proxy.** Papers note that higher coverage does not strongly predict bugs found, so the evaluation should report oracle checks and bugs, not only coverage.

7. How to run things

All commands assume the work folder is set. Put this in the shell first:

```
export FUZZ_WORK=/home/giraffe/cs5187
cd ~/term-project/src
PY=$FUZZ_WORK/.venv/bin/python
```

Everyday commands

I want to...	Command
Check the tools work	$PY harness/smoke_test.py
Run the whole pipeline on 100 seeds	$PY harness/fuzz.py all --limit 100 (about 10 minutes)
Read the report	less $FUZZ_WORK/results/REPORT.md
Triage one failing script by hand	$PY harness/triage.py crash path/to/script.sql
Run the verification bug hunt	$PY harness/hunt.py --workers 4 (takes about an hour)
Install / refresh the dev wheel	$PY harness/devcheck.py --install --version
Check a script on the dev build	$PY harness/devcheck.py --run script.sql
Coverage report for chosen folders	$PY coverage/cov.py report src/optimizer src/execution
Run the LLM client self-test	export LLM_BASE_URL=http://127.0.0.1:11434/v1 LLM_MODEL=qwen2.5-coder:1.5b; $PY llm/client.py
Run guided / unguided LLM experiment	$PY llm/loop.py --run G1 --rounds 4 --prompts 3 --per-prompt 8 ; same with --unguided
Run all tests of the LLM code (no model needed)	$PY llm/test_generate.py ; $PY llm/test_loop.py
Start the local model server	~/ollama/bin/ollama serve (leave it running)

A 30-second live demo for the meeting

```
# on v1.5.6 (with sanitizers) the known ASOF bug crashes:
cd $FUZZ_WORK/duckdb && ../build-asan/duckdb :memory: < ASOF_repro.sql
# on the pinned main build it runs clean, i.e. "already fixed upstream"
```

Use the reproducer in src/docs/week1-n2-validation.md (case A). It demonstrates the pipeline and the value of automatic duplicate detection in one step.

8. Preparing for the progress meetings

Meeting 1 checks three things only: scope is firm, the target is accessible, and an initial pipeline is running. You do not need to present everything. You do need to be accurate about what you do say.

8.1 Five-minute outline

Time	Say	Key numbers
0:30	Problem: random SQL fuzzers mostly produce SQL DuckDB rejects, so deep code stays unexercised.	about half of random mutants fail to bind
1:00	Our pipeline: seeds + hints + coverage gaps -> LLM -> validity check -> sanitizer/coverage builds -> oracles -> automatic triage (draw the diagram from section 2).	
1:30	What exists: 2,560 seeds, 24,816 hints, 5 schemas, coverage tooling, one-command pipeline, auto-triage with upstream duplicate check, bug-hunt tool.	42,637 statements; 80.9% of hints never run
1:00	Early results: real bugs found in the release, all already known; this proves the method and shows why we must target the latest code.	4 real bugs, 0 new
1:00	LLM pilot and plan: small-model results, then the next experiments (requirement-style prompts, mutation, latest-main hunt, OSS-Fuzz baseline).	57% vs 76% valid; 0/10 targets

8.2 Likely questions and honest answers

Question	Answer
Have you found a bug?	Real bugs in v1.5.6, but all were already reported or fixed upstream, so none is new. That is why we now target the newest code.
Why would you find a new one in DuckDB?	It is heavily fuzzed, so odds are modest and we say so. Our edge is directed generation at code the other fuzzers do not reach, DuckDB's own verification oracle on mutated regression tests, and wrong-answer oracles. A negative result with good measurements is still reportable.
What is new compared with existing work?	Quover, MIST, WhiteFox and SmartFuzz exist; we cite them as related work. We combine static hints, coverage-guided LLM generation, a sanitizer build and an automatic upstream-duplicate gate on DuckDB.
Why is the LLM not helping yet?	1.5B model, high temperature, and we gave it raw C++. Published ablations show requirement summaries and few-shot examples matter most. That is the next experiment.
What does triage mean here?	Automatically reproducing a suspect, shrinking it, discarding known harmless patterns, and searching upstream for the same bug.
What is a sanitizer build?	DuckDB compiled with checks that stop and explain on memory misuse, so silent memory bugs become visible crashes.
How do you know a finding is real?	It reproduces alone, on a stock build, shrinks to a few statements, and is not an expected behaviour or a duplicate (we search issues and PRs).
How are you validating the oracle isn't blind?	We run control queries that must fail; this is how we found that verification is inert on the dev wheel.
Who wrote what? Did you use AI?	State the split from section 5 and disclose that much of the harness and LLM code was written with Claude under Prabhnoor's direction. Each person explains their files.
What is the plan to get an accepted bug?	Hunt latest main with sanitizers and verification; improve the generator; mutate regression tests; report any confirmed new finding with a minimal reproducer and root cause.

8.3 Guardrails

- Choose what to emphasise, but do not claim a bug or an LLM win we do not have.
- If asked something you do not know, say what you would check; do not guess.
- Do not report anything upstream before the whole team agrees and a duplicate search is done.
- Disclose AI assistance in the meeting and in the report.

8.4 Ten-minute self-check (each person)

- Explain the pipeline diagram without looking.
- Explain what ASan and the coverage build each tell us.
- Explain the differential oracle and give one example.
- Explain why a bug found in v1.5.6 is not automatically a new bug.
- Explain triage in five steps.
- Name one thing your own part still does not do.

9. Plan, risks and open decisions

9.1 Next steps, in order

#	Step	Who	Why
1	Hunt the latest main with sanitizers and verification on (Niket's pinned build, or a fresh build here)	Prabhnoor / Niket	Only real route to a bug that is not already fixed
2	Replace the generator prompt: plain-language requirement per target, low temperature, few-shot with queries that covered the target, bandit selection	Prabhnoor	Largest effect in the literature
3	Mutation arm: edit queries that reach near a target (regression tests first)	Prabhnoor	Easier task for a small model; bugs cluster near old bugs
4	OSS-Fuzz and SQLsmith baselines at equal time budget	Niket	Promised in the proposal; needed for RQ1
5	Finish noise gates (determinism check, setup taint, unordered windows, quick mode for timeouts)	Niket	Without them every run needs a human to sift noise
6	NoREC / TLP oracles	Niket	Wrong-answer bugs are our best realistic target
7	Evaluation harness with equal budgets and plots; fill paper/data and HANDOFF.md	Prabhnoor / all	Report needs real numbers
8	Reconcile reference environment in the paper (Ubuntu 24.04 vs macOS)	all	Numbers come from different machines

9.2 Risks

Risk	Likelihood	Mitigation
No new bug is found (DuckDB heavily fuzzed)	Medium-high	Target latest main; wrong-answer oracles; report a strong negative result with measurements (partial credit)
Small local model too weak	High	Better prompts; use a stronger model on a teammate's or lab machine; report as a limitation
N3-N6 slip	Medium	Prioritise N4 and N8; move tasks to Jahnvi who has capacity
Time: report and final talk are close	Medium	Start paper from paper/data now; keep HANDOFF.md current
Machine limits (8 GB RAM, no GPU)	Certain	Builds at -j4; heavy LLM work off this machine

9.3 Decisions waiting for the team

- Where to run the latest-main hunt: Niket's existing build, or build here (about 1.5 hours of CPU, much RAM).
- Whether to move N4 or N5 to Jahnvi to even out the load.
- Whether to push the pending local commit (dev-build check) to GitHub.

10. Appendix

10.1 Upstream items referenced

Item	What it is
duckdb/duckdb issue #26300	SIGSEGV on ASOF join with a non-comparison condition (optimizer disabled). Closed.
PR #26301	Estimate the ASOF join cardinality before planning its children. Merged 2026-10-08. Fixes #26300.
PR #26715	Fix MERGE WHEN NOT MATCHED BY SOURCE when the RIGHT join is not flipped. Merged 2026-10-12.
Issues #5984, #7128	'Fuzzer issues' umbrella lists from 2023 that contain the COLLATE-in-BETWEEN printing failure.
PR #24411	Rewrite IS DISTINCT FROM NULL. Probably related to the optimizer short-cut we saw on an out-of-range date; we did not verify the link.
Issue #26292 (referenced in #26300)	Upstream introduced a disabled-optimizer CI run, which is why that vein is closing quickly.

10.2 Related work we have read (abstract level unless stated)

Paper	Idea we can use
WhiteFox (OOPSLA 2024, arXiv 2310.15991)	Two agents: analyse optimizer source into requirements, then generate tests; feedback via triggering tests as few-shot examples chosen by a bandit. Requirements beat raw code (1,113 vs 638 triggering tests).
MIST (ICSE 2026, arXiv 2603.21530)	Generate from a feature tree, then mutate seeds with Monte Carlo tree search after coverage plateaus. Evaluated on DuckDB with 7B-32B models; reports coverage, no bugs.
Quover (DBTest 2026)	Closest to ours: find uncovered functions in the DBMS, give an LLM their code, generate targeted SQL.
SmartFuzz (OOPSLA 2026)	Seeds from documentation and past crash-triggering inputs; 61 new DBMS bugs reported.
CovRL (ISSTA 2024, arXiv 2402.12222)	LLM mutators can lower diversity and coverage; weight rewards by rare coverage.
SQLancer / SQLancer++, QPG	Oracles: TLP, NoREC, CERT, differential query plans; query plan guidance as a cheap feedback signal.
ParserFuzz (arXiv 2503.03893)	Derive generation from the DBMS's own grammar and track grammar-rule coverage.
HGFuzzer (arXiv 2505.03425)	Directed fuzzing: regenerate until coverage confirms the target was reached.

10.3 What has not been verified

- Jahnvi's RQ3 numbers and the hint counts are the teammate's outputs; I re-counted the hint totals from hints.json but did not re-run the coverage analysis.
- Niket's build timings were measured on a MacBook (M1, 16 GB), a different machine from ours; the single-file timings are not whole-build times.
- Conclusions from the LLM pilot rest on 4 rounds, 3 prompts per round, one seed and a 1.5B model; one round dominates the unguided total.
- Literature numbers are the authors' claims from abstracts or partial reading, not reproduced.
- The latest main has not been hunted with verification and sanitizers on this machine; 'no new bug' applies to v1.5.6 only.

- About a quarter of the v1.5.6 hunt runs (mostly later seeds) were not executed because the run was stopped.

10.4 Disclosure note (copy into the report)

Substantial parts of the code in src/harness, src/coverage and src/llm, and the analysis in this document, were developed with the assistance of Claude (Anthropic) working under the direction of the first author. Each team member reviewed and can explain the files attributed to them. Teammates should add their own disclosure of any AI assistance for src/static, src/gen, build scripts and documents they wrote.